

The background features a pattern of hexagons in various sizes and colors. Some hexagons are solid (blue, red, yellow), while others are light gray outlines. They are arranged in a way that some overlap, creating a geometric, honeycomb-like structure.

ADD (Attribute-Driven Design) 01





Recap

▶ Requirement Capture

- Goal: Use Case Modeling and Requirement Capture
- Use Case Modeling
 - Input: Business Modeling Artifacts
 - Output: Use Case Diagram, Use Case (Extended) Description
- Requirements Capture
 - Input: Business Modeling & Use Case Modeling Artifacts
 - Output:
 - Functional Requirement Description
 - Non-Functional Requirement Description including Quality Requirements

▶ OO Functional Analysis

- Goal:
 - OO Functional Design을 위한 Problem Domain Analysis
- Input
 - Use Case Modeling Artifacts and Functional Requirement Description
- Output
 - Conceptual Modeling Diagram and Description



Recap

▶ OO Functional Design

- System Behavior Description by System Interaction Diagram
- Identify system external events for each UC
- Contract description for system operations
 - Responsibility
 - Post-condition
- Interaction Diagrams
 - Draw collaboration diagram for each system operation applying GRASP
 - Ex) enterItem operation
- Class Diagrams



Contents

- ▶ Designing Software Architecture
- ▶ Architecture Drivers
- ▶ Design Strategies



ADD Motivation

- ▶ How to design software architecture in a systematic, predictable, repeatable, and cost-effective way
 - By meeting stakeholder needs, adhering to budgets and schedules, dealing with constraints
- ▶ If you are building a complex system, then chances are that you are trying to balance many competing forces.
 - time to market, cost, performance, evolvability, usability, availability, and so on.



Attribute-Driven Design (ADD) Method

- ▶ Developed by the Carnegie Mellon[®] Software Engineering Institute (SEI)
- ▶ A systematic recursive decomposition process for designing the software architecture
 - decomposes a system or system element
 - by applying architectural styles and tactics that satisfy its architecture drivers



A Brief History of ADD

- ▶ The first version of ADD (ADD 1.0)
 - originally called ABD (“Architecture-Based Design”) was published in January 2000
- ▶ The second version (ADD 2.0) in November 2006.
 - architect to use and combine tactics and patterns to achieve the satisfaction of quality attribute scenarios
 - iterations are always driven by the selection and decomposition of architectural elements.
- ▶ The third version (ADD 3.0) in 2016
 - promotes quick design iterations in which a small number of design decisions are made, potentially followed by an implementation spike.
 - explicitly promotes the (re)use of reference architectures and is paired with a “design concepts catalog”,
 - includes a broad selection of tactics, patterns, frameworks, reference architectures, and technologies

Designing SA by ADD

Principle

Tactic

Style

RA

External
Component

Practical
Examples

Design Purpose

Quality Attributes

Primary Functionality

Architectural Concerns

Constraints

Architectural Drivers

Design Concepts

<<selects and
instantiates>>

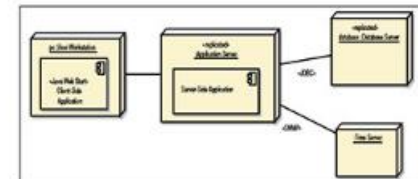
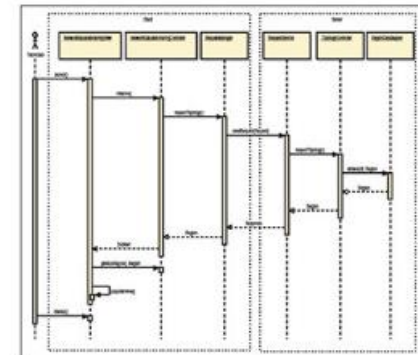
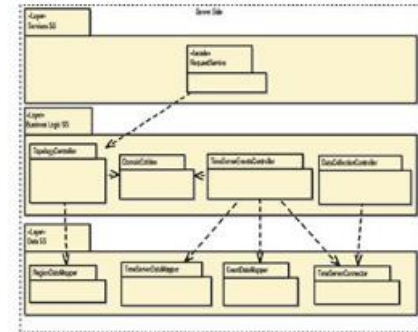
<<uses>>



ADD Methodology

The Architect

<<produces>>



(Documented) Structures
resulting from
design decisions

The background features a collection of hexagons in various sizes and colors, including gold, grey, and light blue, scattered across the slide. A large, prominent gold hexagon is positioned in the upper left quadrant, while a large grey hexagon is on the right side. The title 'Architecture Drivers' is centered in a bold, gold font.

Architecture Drivers



Architecture Drivers (Inputs)



- ▶ Design Purpose
- ▶ Quality attributes
- ▶ Primary Functionality
- ▶ Architectural Concerns
- ▶ Constraints

→ System stakeholders have prioritized according to business and mission goals.



Design Purpose

► Design Purpose

- Which business goals is the organization most concerned about at this time?
 - For a **project proposal**
 - For as part of the process of creating an **exploratory prototype**
 - For designing your architecture **during development**
 - In a **mature domain** or In a **novel domain**
- ***Before*** beginning the design process, the architect should have **clear design goals** and establish a **clear design purpose**.



Quality Attributes

- ▶ Quality attributes are an essential part of the **nonfunctional requirements of a system**.
- ▶ **Measurable or testable properties of a system** that are used to indicate how well the system satisfies the needs of its stakeholders
- ▶ Among the drivers, quality attributes are the ones that **shape the architecture the most significantly**
- ▶ The responsibility of system analysts **put together a complete list of quality attributes before the detailed design process can begin.**
- ▶ A architect must worry about eliciting, specifying, **prioritizing**, and validating quality attributes

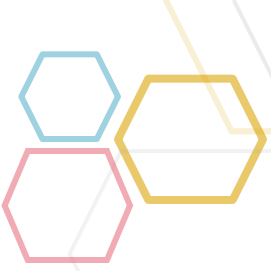
Quality Attributes





Primary Functionality

- ▶ Primary Functionality
 - **Critical to achieve the business goals** that motivate the development of the system
 - **Having high level of technical difficulty**
 - Requiring the **interaction** with many architectural elements.
- ▶ As a rule of thumb, **approximately 10 percent** of the given use cases or user stories are likely to be primary.



Primary Functionality

- ▶ **Allocation of functionality to elements(modules)** is what matters in architecture design.
 - A good architecture **can promote modifiability or reusability**, and also plan work assignments.
 - **Bad decisions can be paid through the use of refactoring**, although this impacts the project's rate of progress, or velocity
- ▶ Some **quality attribute scenarios**(e.g. performance) are **directly connected to** the primary functionality in the system, but some are not(e.g. availability).



Architectural Concerns

- ▶ Architectural Concerns
 - Aspects to be considered but not expressed
- ▶ **General concerns:** a broad issues in creating the architecture
 - establishing an overall system structure
 - allocation of functionality to modules
 - allocation of modules to teams
 - organization of the code base
 - startup and shutdown, and
 - supporting delivery, deployment, and update



Architectural Concerns



- ▶ **Specific concerns:** detailed system-internal issues
 - exception management, dependency management,
 - configuration, logging,
 - authentication, authorization,
 - caching,
 - and so forth that are common across large numbers of applications
- ▶ **Internal (derived) requirements**
 - aspects that facilitate development, deployment, operation, or maintenance of the system
- ▶ **Issues**
 - result from analysis activities



Architectural Concerns



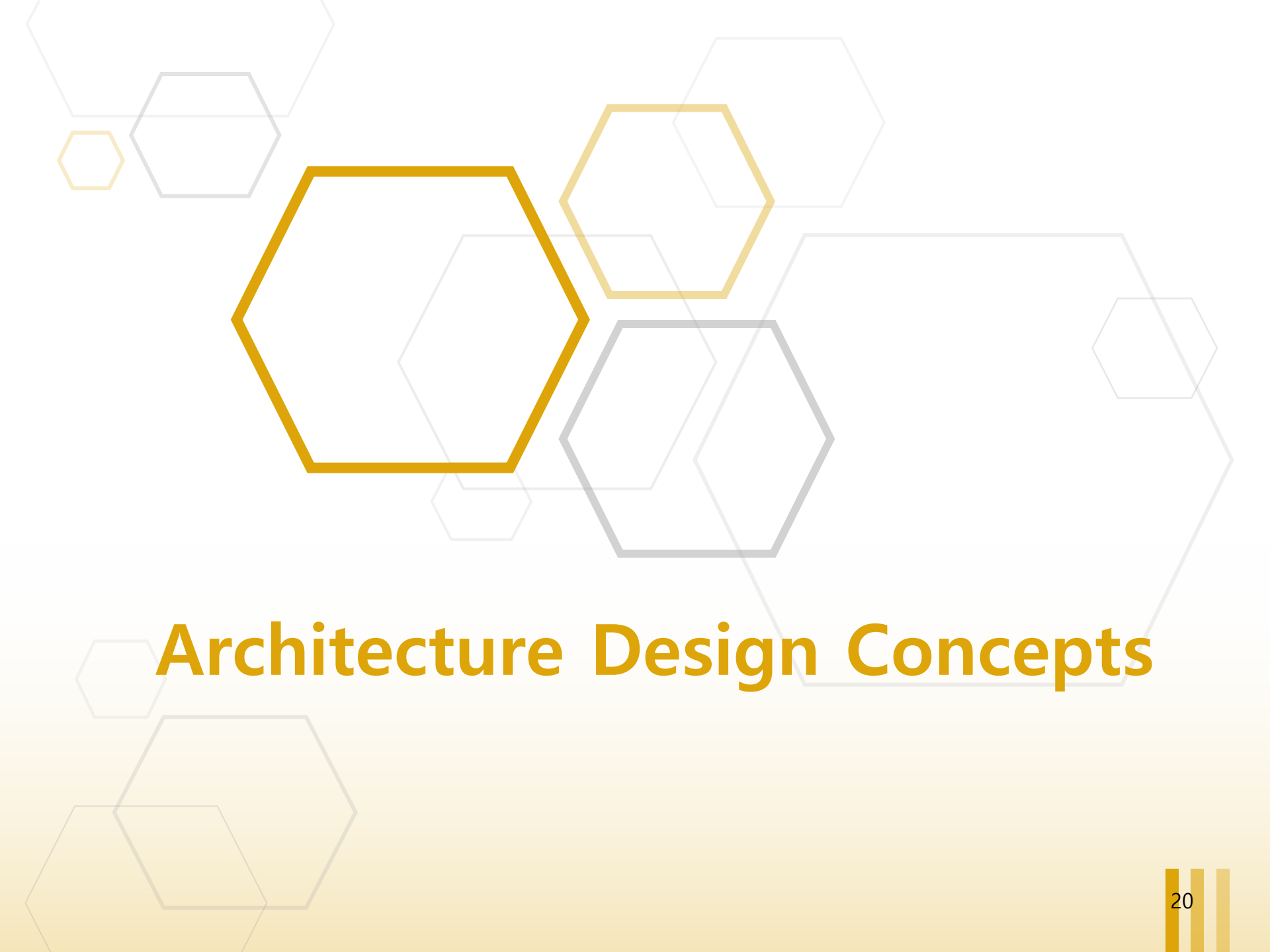
- ▶ **From their past experience,**
 - **wise architects** are usually **aware of** the concerns that are associated with a particular type of system and the need to make design decisions to address them.
 - **Inexperienced architects** are usually less aware of such concerns.
- ▶ Architectural concerns frequently result in the introduction of **new quality attribute scenarios**.



Constraints

▶ A **constraint**

- a decision over which you have little or no control as an architect.
- Examples
 - mandated technologies,
 - other systems with which your system needs to interoperate or integrate,
 - laws and standards that must be complied with,
 - the abilities and availability of your developers,
 - deadlines that are non-negotiable,
 - backward compatibility with older versions of systems,
 - use of open source technologies,
 - and so on.



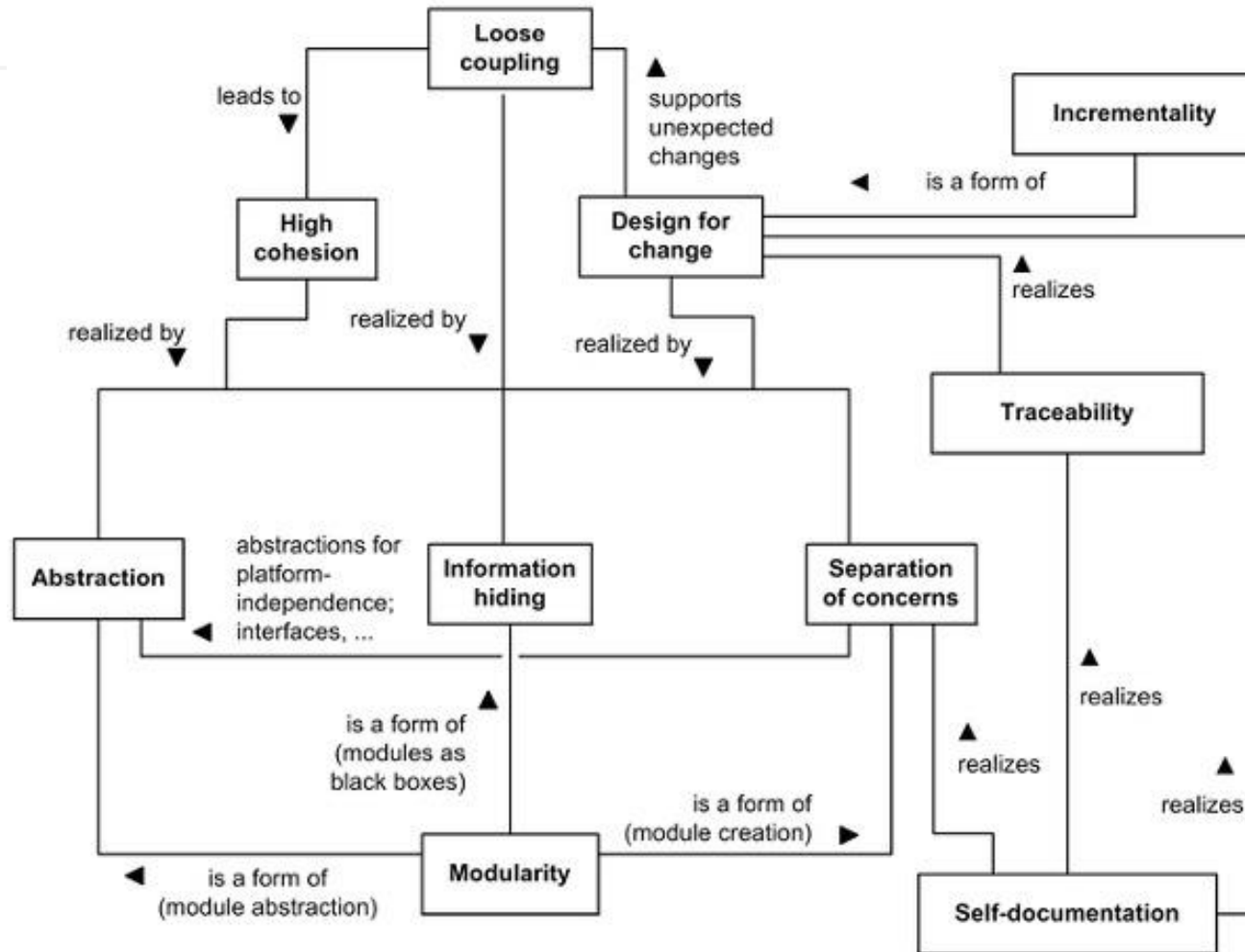
Architecture Design Concepts



SA Design Strategies

- ▶ SW Design Principles
 - For Flexibility and Reusability
- ▶ SW Architecture Tactics
 - To achieve SW Qualities
- ▶ SW Architecture Styles(Patterns)
 - To build well-proven SW Architectures
- ▶ Reference SW Architectures
 - To build well-proven SW Architectures for a specific domain
- ▶ Externally Developed Components
 - to make design concepts concrete and closer to the actual implementation

SW Design Principles



[Ref: Software Architecture - A Comprehensive Framework and Guide for Practitioners]



SOLID, GRASP Design Principles

► **SOLID:**

- Five design principles intended to make OO designs more **understandable, flexible, and maintainable**

► **SOLID Principles**

- **S**ingle Responsibility Principle
- **O**pen Closed Principle
- **L**iskov Substitution Principle
- **I**nterface Segregation Principle
- **D**ependency Inversion Principle

► **GRASP**

- Fundamental design principles for **reusable and changeable** by
 - assigning responsibilities to classes
 - determining the classes that will be in a design

► **GRASP Principles**

- Low Coupling
- High Cohesion
- Expert
- Creator
- Controller
- Don't talk to Stranger



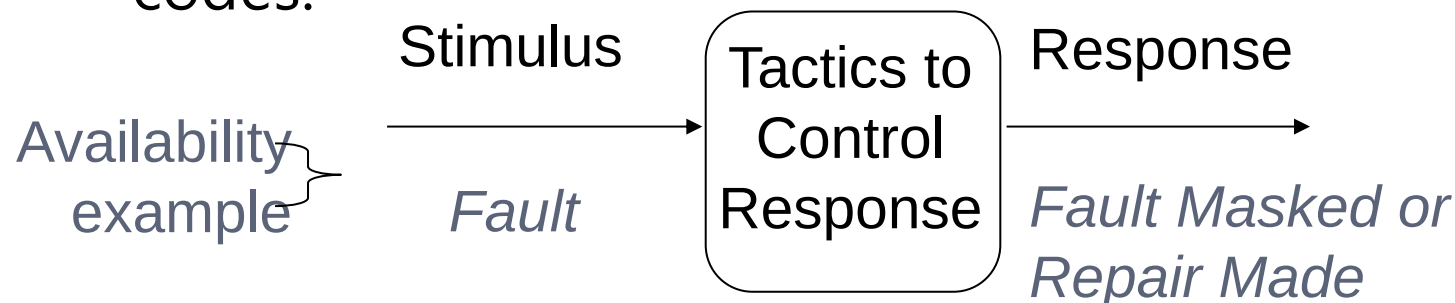
Architecture Tactics

► Architecture Tactics

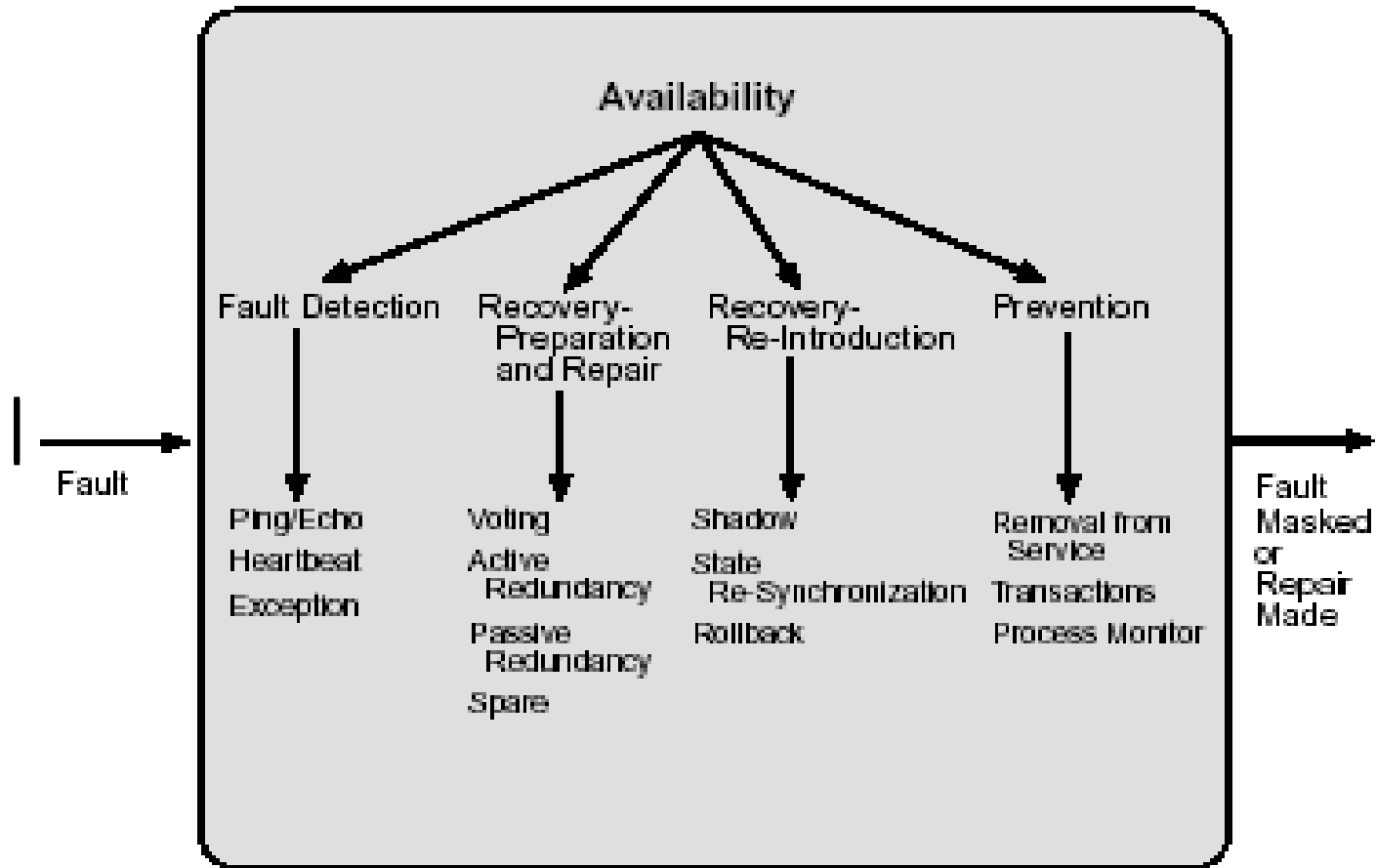
- Collections of **fundamental design techniques(primitives)** to achieve a response for particular quality attributes
- Tactics are both simpler and more primitive than styles
 - Tactics focus on the **control of a single quality attribute response**.
 - Cf) Styles, in contrast, typically focus on resolving and balancing multiple forces, i.e., multiple quality attribute goals.

Architecture Tactics

- ▶ Tactics provide a **top-down** way of thinking about design.
- A tactics categorization begins with a set of design objectives related to the **achievement of a quality attribute**, and presents the architect with **a set of options** from which to choose.
- These options then need to be further instantiated through some combination of styles, frameworks, and codes.



Availability Tactics

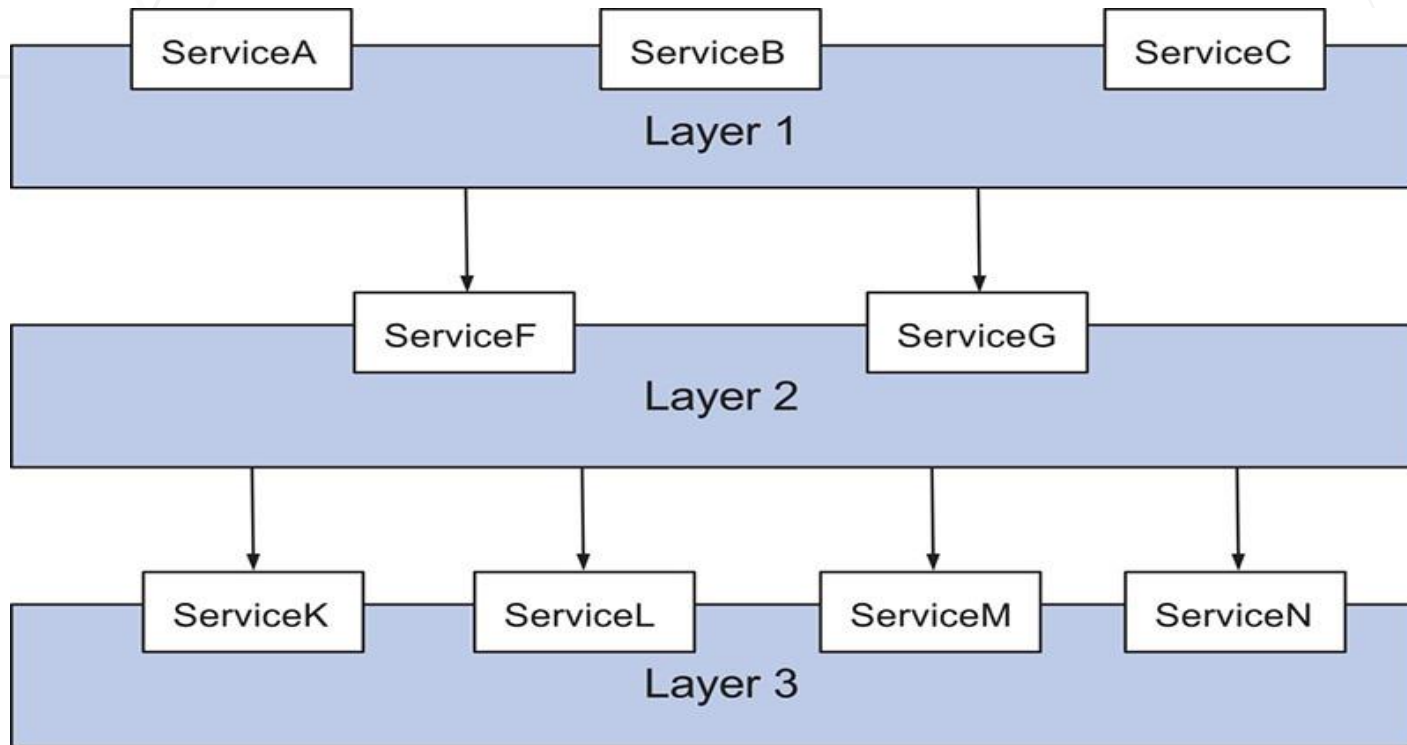




Architecture Style (Pattern)

- ▶ Architectural Style
 - fundamental **structure of a software system** and **its properties** to solve recurring design problems in a defined context
 - define types of components(elements) and connectors in a specified topology
 - useful for structuring an application either logically or physically

ex) Layered Architecture Style



Legend:

Layer

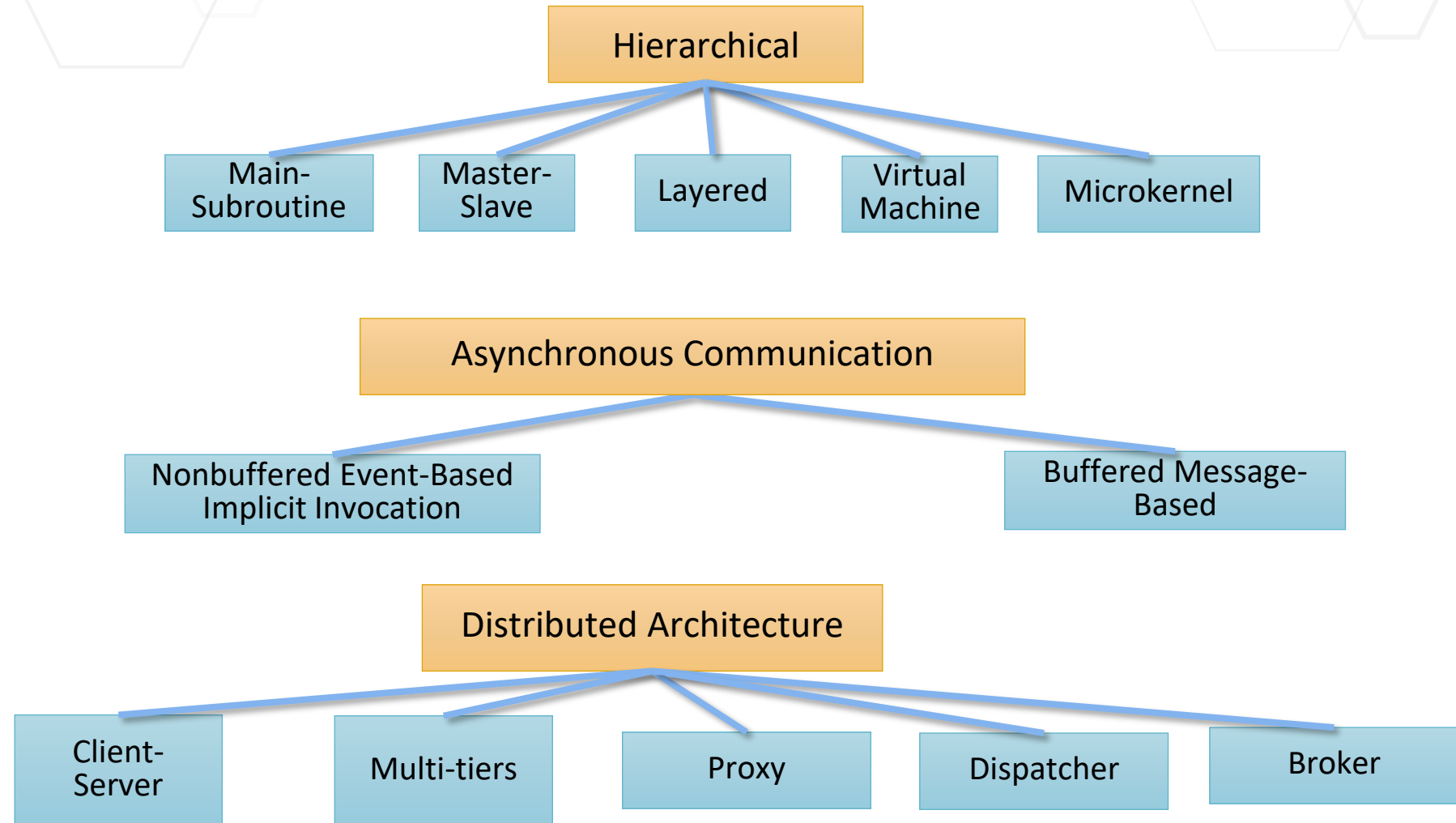
Service Interface

Calls/Invokes



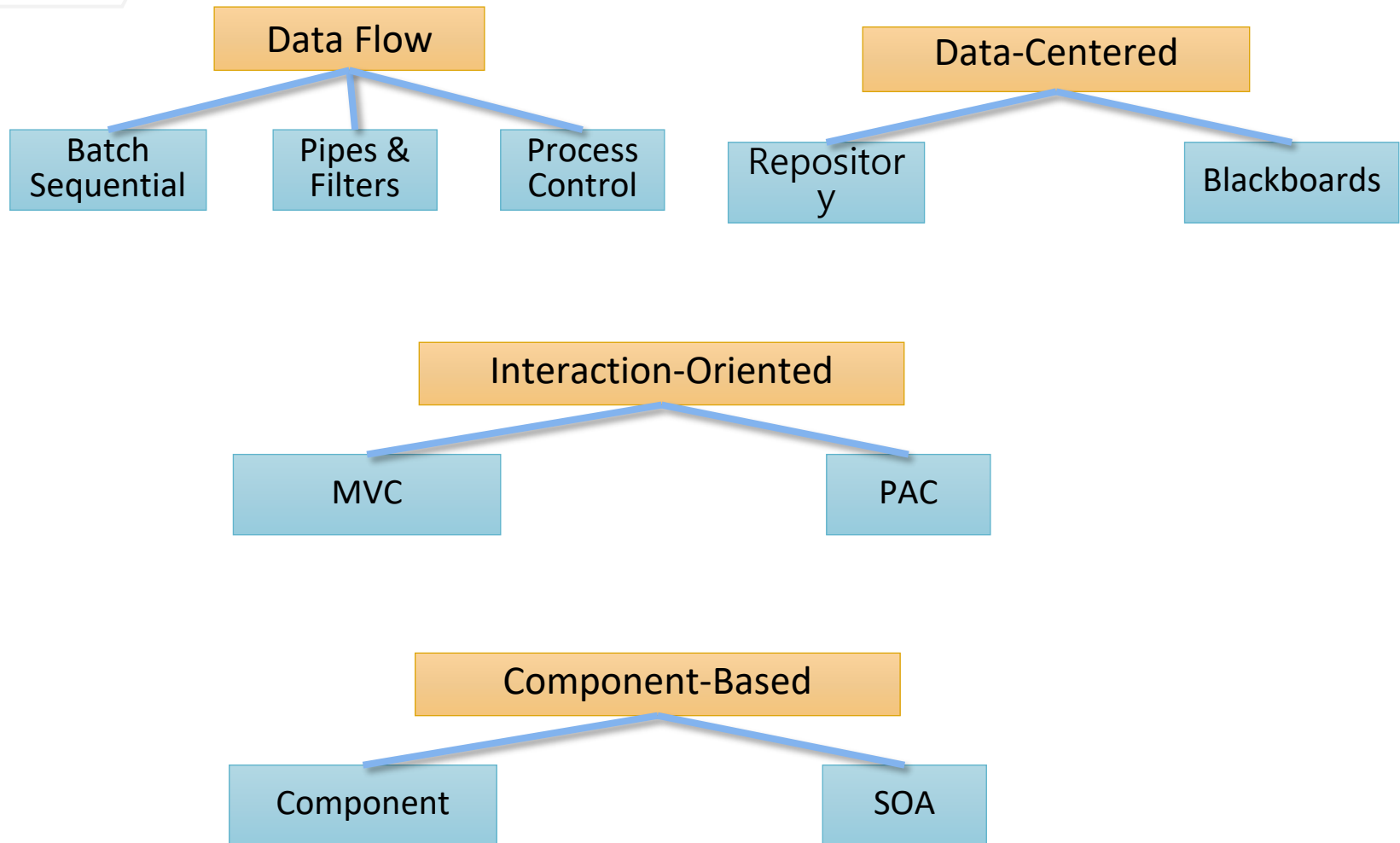
A Taxonomy of Architecture Style

- Lecture Contents 1 -



A Taxonomy of Architecture Style

- Lecture Contents 2 -





Deployment Styles

- ▶ Deployment Architecture Styles
 - provide models on how to physically structure the system to deploy it.
- ▶ Examples
 - Non-distributed Deployment
 - Distributed Deployment
 - 2-Tier Deployment (Client-Server)
 - 3-Tier Deployment
 - 4-Tier Deployment
 - Performance Styles
 - Load-Balanced Cluster



Modern SA Architecture Styles

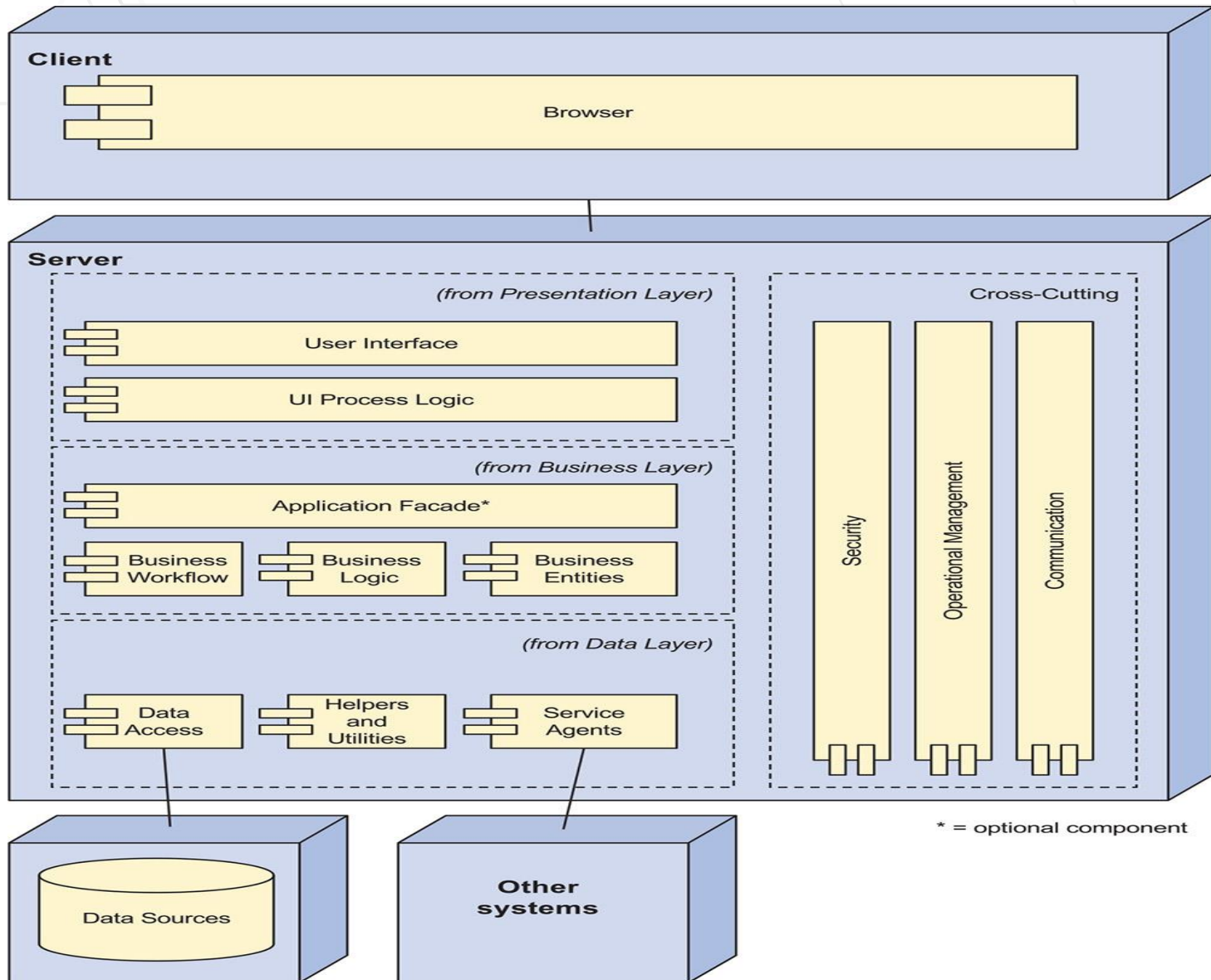
- ▶ Peer-To-Peer Architecture
- ▶ Publish/Subscribe Architecture
- ▶ Middleware Architecture
- ▶ Component Platforms
- ▶ Service Oriented Architecture
- ▶ Security Architecture
- ▶ Cloud Computing Architecture



Reference Architectures

- ▶ Reference Architectures
 - blueprints that provide an overall logical structure for particular types of applications
- ▶ Examples
 - Web Application RA
 - Rich Client Applications RA
 - Rich Internet Applications RA
 - Mobile Applications RA
 - Service Applications RA

Ex) Web Application RA





Reference Architecture vs. Architecture Style



- ▶ Reference architectures vs. architectural styles
 - Architectural styles
 - define types of components and connectors in a specified topology
 - useful for structuring an application either logically or physically
 - technology and domain agnostic
 - Reference architectures
 - provide a structure for applications in specific domains
 - embody various different styles



Externally Developed Components

- ▶ Externally Developed Components
 - When you are designing a software architecture, you need to make these design concepts concrete and closer to the actual implementation.
 - There are **two ways to achieve** this:
 - You can code the elements obtained from tactics and patterns (**build**)
 - you can associate technologies with one or more of these elements in the architecture (**buy**)
 - This *buy versus build* choice is one of the most important decisions you will make as an architect.



Few criteria when selecting externally developed components 1

- ▶ *Problem that it addresses.*
 - Is it something specific, such as a framework for object-oriented to relational mapping or something more generic, such as a platform?
- ▶ *Cost.*
 - What is the cost of the license and, if it is free, what is the cost of support and education?
- ▶ *Type of license.*
 - Does it have a license that is compatible with the project goals?
- ▶ *Support.*
 - Is it well supported? Is there extensive documentation about the technology? Is there an extensive user or developer community that you can turn to for advice?
- ▶ *Learning curve.*
 - How hard is it to learn this technology? Have others in your organization already mastered it? Are there courses available?



Few criteria when selecting externally developed components 2

▶ *Maturity.*

- Is it a technology that has just appeared on the market, which may be exciting but still relatively unstable or unsupported?

▶ *Popularity.*

- Is it a relatively widespread technology? Are there positive testimonials or adoption by mature organizations? Will it be easy to hire people who have deep knowledge of it? Is there an active developer community or user group?

▶ *Compatibility and ease of integration.*

- Is it compatible with other technologies used in the project? Can it be integrated easily in the project?

▶ *Support for critical quality attributes.*

- Does it limit attributes such as performance? Is it secure and robust?

▶ *Size.*

- Will the use of the technology have a negative impact on the size of the application under development?



Summary

- ▶ Designing Software Architecture
- ▶ Architecture Drivers
 - Design Purpose
 - Quality attributes (= System Properties)
 - Primary Functionality
 - Architectural Concerns
 - Constraints
- ▶ Design Concepts
 - SW Development Paradigms
 - SW Design Principles (SOLID, GRASP)
 - SW Architecture Tactics
 - SW Architecture Styles(Patterns)
 - Reference SW Architectures
 - External Components
 - Practical Examples

The background features a light gray hexagonal grid. Several hexagons are highlighted with colored outlines: a red one in the upper left, a yellow one below it, a large blue one in the center, and a small light blue one in the upper right. A small yellow hexagon is also at the bottom center.

Thank You!

